

Project Report Template

Advanced Analytics and Applications

Team 07



Authors:

Larissa Theilmann (7432976)

Emre Uyar (7443638)

Tjorge Orlitz (7443662)

Nils Hornstein (7369566)

Contact: torlitz@smail.uni-koeln.de

Supervisor: Univ.-Prof. Dr. Wolfgang Ketter

Co-Supervisor: Janik Muires

Chair of Information Systems for Sustainable Society
Faculty of Management, Economics and Social Sciences
University of Cologne

2026-04-01

Table of contents

1	Introduction	1
1.1	Motivation and Problem Statement	1
1.2	Research Questions	1
1.3	Report Structure	2
2	Data Description	2
2.1	Data Sources	2
3	Data Preparation	3
3.1	Taxi Trip Description and Cleaning	3
3.2	Weather Data	3
3.3	Points of Interest	3
3.4	Spatial and Temporal Discretisation	4
3.5	Feature Engineering	4
4	Data Analytics	4
4.1	Descriptive (Spatial) Analytics	4
4.1.1	Demand Patterns	4
4.1.2	Spatial Patterns	4
4.1.3	Temporal Patterns	4
4.1.4	Effect of Spatio-Temporal Resolution	4
4.1.5	Demand Hot Spots (GMM / Kernel Density)	4
4.1.6	Weather Effects	4
4.2	Predictive Analytics (SVM & Neural Networks)	4
4.2.1	Problem Setup and Validation Strategy	4
4.2.2	Support Vector Machine	5
4.2.3	Deep Learning Model	8
4.2.4	Model Comparison	8
4.3	Smart Charging (Reinforcement Learning)	8
4.3.1	Problem Formulation (MDP)	9
4.3.2	Environment (Discrete-Event Simulation)	9
4.3.3	Q-Learning Agent	9
4.3.4	Training	9
4.3.5	Learned Policy	9
4.3.6	Evaluation	10
5	Discussion and Outlook	10
5.1	Summary of Findings	10
5.2	Implications for the Client	10
5.3	Limitations	10
5.4	Future Work	10
	References	10

List of Figures

1	Direct (3.2a) vs. residual-on-baseline (this notebook) SVR skill, both independently tuned, per level.	7
2	Learned Charging Policy	9

1 Introduction

1.1 Motivation and Problem Statement

The ride-hailing market is already crowded. Uber, Lyft, FreeNow, Lime, and Tier compete for the same trips, and current market reports point to ride-hailing, increasingly paired with autonomous and electric vehicles, as the segment that will dominate urban mobility in the near term. Our client, a well-established German car manufacturer, does not intend to watch from the sidelines. It is building a ride-hailing platform on a fully electrified vehicle fleet and plans to launch first in the United States, where customers have proven more receptive to novel mobility business models.

The gap is not ambition. It is operational know-how. The client can design and build vehicles; it has never run a ride-hailing fleet, and it has never run an electric one, where charging schedules constrain vehicle availability in ways a conventional taxi or rental fleet never had to manage. Before it can plan routes, size a fleet, or choose where to place its first vehicles, the client needs a working account of how demand for hailed rides moves through a city across space and time, and no such account yet exists for the market it plans to enter.

This report builds that account from historical taxi trip data for Chicago, made available by the Chicago Data Portal and covering trips from 2024 onward. Taxis are not ride-hailing vehicles, but in a market the client has not yet entered, they are the closest available proxy for one: metered urban trips, dispatched on demand, subject to the same weather and time-of-day patterns that will govern the client's own fleet. We supplement the trip records with independently collected weather data and, where it adds value, point-of-interest data from OpenStreetMap, and we analyze demand at spatial resolutions ranging from coarse census tracts to fine-grained H3 hexagon grids, and at multiple temporal resolutions, to establish which level of detail the data can actually support.

Two decisions sit downstream of this analysis and give the report its structure. The first is where and when to expect demand, at a resolution fine enough to plan a fleet against rather than merely describe a market. The second is specific to an electric fleet: how a vehicle should charge at a driver's home between shifts, given a battery of finite capacity and a next-shift energy requirement that is not known with certainty in advance. Both decisions are considerably cheaper to get right on paper, using historical data and simulation, than after the client has already committed to vehicles and charging infrastructure. Getting the second one right also matters for the reason the client chose an electric fleet in the first place: a fleet only delivers on that choice if it is charged reliably enough to be available when a ride is requested.

1.2 Research Questions

The analysis that follows is organized around five questions, each feeding into the next.

1. How does taxi demand vary across space and time in Chicago, and what plausibly explains the patterns observed at different spatial units (census tract versus H3 hexagons at resolution 7 and 8) and temporal bin sizes?
2. Where are the persistent spatial hot spots of demand, and what do Gaussian mixture models and kernel density estimation reveal about their structure?
3. Can demand be predicted for a specific spatial unit and time bucket rather than simply extrapolated from the previous hour, for instance answering what demand to expect on a rainy Sunday between 10 and 11 a.m. on the outskirts of the city, and how does a support vector machine compare against a feedforward neural network on this task?

4. What charging policy should an electric taxi driver follow at home, given a charging rate that can be adjusted every fifteen minutes, a battery capacity that cannot be exceeded, and a next-shift energy requirement that is only known probabilistically at the moment the driver departs?
5. What do these findings imply for the client’s fleet operations, and in particular, should the client invest in private home charging infrastructure or rely on public charging networks instead?

1.3 Report Structure

The remainder of this report is organised as follows. Section 2 describes the data sources. Section 3 describes the preparation pipeline. Section 4 presents the descriptive spatial analysis, the demand-prediction models, and the reinforcement-learning approach to smart charging. Section 5 discusses implications for the client and outlines future work.

2 Data Description

2.1 Data Sources

Four raw datasets underpin the analysis: taxi trip records, hourly weather observations, points of interest, and a boundary layer of Chicago’s community areas. Each is described below by origin, coverage, and native granularity; the cleaning and feature-engineering steps applied to each follow in Section 3.

The taxi trip records come from the Chicago Data Portal’s Taxi Trips dataset. Before use, invalid rows (missing key identifiers, non-positive trip duration, or negative distance or fare) were dropped, and the table was trimmed to sixteen fields: trip and taxi identifiers, start and end timestamps, trip duration and distance, pickup and dropoff census tract and community area codes, the fare total, the operating company, and pickup and dropoff coordinates. Because census tract is missing from just over half of all trips (a mix of privacy suppression and trips ending outside Chicago) while community area is missing from under 9%, the cleaned data is kept as two parallel datasets: 14,494,804 trips with a known community area, and a stricter 6,835,451 trips that also have a known census tract. Both span January 1, 2024 through May 31, 2026, and each trip is a dispatched, metered ride at whatever time and location a passenger requested it, which is the pattern the client’s own fleet will eventually have to anticipate.

Weather observations come from the ERA5-Land reanalysis product, pulled through the Copernicus Climate Data Store API for a single coordinate near Chicago (latitude 42, longitude -87.8) rather than a network of ground stations. The record runs from January 1, 2024 through May 12, 2026 at hourly resolution and carries six variables: 2-metre air temperature, total precipitation, snow cover, snow depth, and the two horizontal components of wind at 10 metres. A reanalysis is a modeled reconstruction rather than a direct station reading, but it gives complete, gap-free hourly coverage for the entire trip period, which no single Chicago weather station could promise.

Points of interest come from OpenStreetMap, extracted with OSMnx for the “Chicago, Illinois, USA” boundary and limited to three tag families: amenity, shop, and tourism. The raw download returns 43,237 points spread across 311 distinct combinations of those tags, later collapsed into fourteen categories relevant to travel demand (food and drink, nightlife, entertainment, and so on) during preparation. Unlike the trip and weather data, this is a snapshot of the map as it stands today, not a historical record tied to the trip period.

The fourth source is a boundary layer for Chicago’s 77 official community areas, used for spatial reference and choropleth mapping throughout the report. It is a distinct unit from the census tracts already present in the trip data itself: the taxi records carry pickup and dropoff census tract and community area codes directly, sourced under the portal’s own privacy rules, while this boundary layer only supplies the outlines needed to draw them on a map.

3 Data Preparation

This section describes the preparation pipeline that turns the raw sources described in Section 2 into the analysis-ready datasets used throughout the report.

3.1 Taxi Trip Description and Cleaning

3.2 Weather Data

3.3 Points of Interest

Point-of-interest data was optional under the assignment brief, but urban-analytics research treats POI density as one of the more reliable proxies for where and when ride demand originates: restaurants and nightlife venues draw evening trips, transit hubs and offices draw commute-time trips, and the mix varies enough by neighborhood to be worth capturing alongside the taxi and weather data.

We extracted points of interest from OpenStreetMap using OSMnx, querying the “Chicago, Illinois, USA” boundary for three tag families: amenity, shop, and tourism. The query returned 43,237 points, and coverage across the three tags is uneven: 7,053 records lack an amenity tag, 37,552 lack a shop tag, and 41,738 lack a tourism tag, so most points carry only one of the three. We combined the three columns into a single `poi_type` field by taking amenity where present and falling back to shop, then tourism; every point ended up with one of 311 distinct raw types, and none were left without a type or without geometry.

At 311 distinct raw types, the data is too granular to use directly as model features, so we imposed a manual mapping from each raw type to one of fourteen broader categories that plausibly drive ride-hailing demand: food and drink, nightlife, entertainment, leisure and sports, grocery, shopping, health, finance, education, lodging, transport, automotive, services, and civic and community. The mapping doubles as a filter. Fifty-five raw types, mostly infrastructure and hyper-local street furniture such as benches, parking entrances, post boxes, waste bins, and device charging stations, do not fit any category and were dropped into an “other” bucket rather than kept as noise. That bucket absorbs 23,129 of the 43,237 points, leaving 20,108 categorized points of interest.

Each retained point was assigned to an H3 hexagon at both resolution 7 and resolution 8 from its centroid coordinates, then aggregated to the hexagon level: a count of points, a count of distinct POI types, and the categorized counts in wide format with one column per category. At resolution 7, 152 hexagons contain at least one point of interest; at resolution 8, that rises to 806 hexagons, since the finer grid packs the same points into a larger number of smaller cells. The resolution-7 counts feed into the feature set used for demand prediction in Section 4.2, alongside POI aggregations at census-tract and community-area level; the resolution-8 aggregation is retained for the spatial exploratory analysis in Section 4.1.

3.4 Spatial and Temporal Discretisation

3.5 Feature Engineering

4 Data Analytics

This section covers all analytical methods applied to the prepared data: descriptive spatial analytics, predictive modelling of demand (SVM and neural networks), and reinforcement learning for smart charging.

4.1 Descriptive (Spatial) Analytics

We explore taxi demand along the spatial, temporal, and weather dimensions to build intuition before modelling.

4.1.1 Demand Patterns

4.1.2 Spatial Patterns

4.1.3 Temporal Patterns

4.1.4 Effect of Spatio-Temporal Resolution

4.1.5 Demand Hot Spots (GMM / Kernel Density)

4.1.6 Weather Effects

4.2 Predictive Analytics (SVM & Neural Networks)

Fleet planning needs to know where and when demand will appear before the shift starts, so we predict trip starts per spatial unit and time bucket from calendar, location, weather, and POI features alone, without extrapolating one hour from the previous one. The question the model answers is a scenario question: how much demand to expect on a rainy Sunday morning on the outskirts. We compare two model families on this task, a Support Vector Machine (fitted both directly on demand and on the residual over a historical-mean baseline) and a feedforward neural network, across six grids that cross three spatial units (H3 resolution 7, census tract, community area) with hourly and daily time bins. Both are scored against the same historical-mean baseline, which is what lets us ask whether a learned model, and a deep one in particular, is worth its added complexity here.

4.2.1 Problem Setup and Validation Strategy

The prediction target is `Total_Trip_Start`, the number of trips beginning in a given spatial unit during a given time bucket. Prediction is scenario-conditioned rather than autoregressive: no model sees the previous bucket's demand, only the cell's identity and location, the calendar, the weather, and the POI counts for that cell. That restriction comes from the assignment brief, and it also matches the client's actual planning problem, which is positioning vehicles ahead of a shift rather than reacting to the hour just past. What each model *fits* is not identical to the target itself, since the residual SVM predicts a correction to a baseline rather than demand directly; the two strategies are described in the subsections below.

Every model is trained on six grids: three spatial units (H3 resolution 7 with 108 active hexagons, 496 active census tracts, and 77 community areas) crossed with hourly and daily bins. Resolution 8 is explored descriptively in Section 4.1 but excluded from modelling [GAP: state why res 8 was dropped before modelling; the pipeline filters it out without recording a reason], and inactive units, those with no pickups anywhere in the observation window, are dropped during grid construction (Section 3). Sparsity differs sharply across the six: the share of zero-demand rows runs from 0.8% at community/daily to 96.5% at census/hourly. That spread drives the sampling decisions taken separately for each model family below.

The reference point throughout is a historical mean per spatial unit, keyed by month and day of week on the daily grids and additionally by hour on the hourly ones. It is computed on the training split alone and merged into validation and test, so it carries no future information, and it serves twice over: as the `base_demand` feature available to both models and as the benchmark for the skill score, $1 - \text{MAE_model} / \text{MAE_baseline}$. Skill rather than raw MAE is reported because mean demand differs by roughly a factor of 80 across the six grids, which makes absolute errors incomparable between them. Both model families fit in `log1p` space and invert with `expm1`, clipped at zero, so every metric quoted here is in count space.

Train, validation and test are cut along the calendar rather than drawn at random. The earliest 50% of unique dates train the models, the next 20% serve for validation, and the final 30% are held back for testing [GAP: state the actual date boundaries of the three splits]. A random row-level split would place adjacent hours of the same cell on both sides of the boundary and leak autocorrelated demand into the test set, and it would also contaminate `base_demand`, which is estimated from training dates only. Model selection, covering feature sets, kernels, architectures, and hyperparameters, is therefore carried out on the validation split, with the test split scored once for the selected models. [GAP: this single-scoring discipline holds for the SVM notebooks; confirm and state plainly how the NN's per-level scores were obtained before finalising, see Section 4.2.3.] [GAP: one EDA notebook argues the final week of collection is incomplete and should be excluded; no such truncation exists in the pipeline. Resolve and state here, or move to the limitations.]

4.2.2 Support Vector Machine

Two SVR strategies are trained on identical splits, samples and candidate feature sets, differing only in what they are asked to predict. The direct model regresses `log1p(Total_Trip_Start)` on the feature block and converts predictions back with `expm1`, clipped at zero. The residual model predicts the log-space correction to the historical mean instead, `log1p(demand) - log1p(base_demand)`, and reconstructs counts as `expm1(log1p(base_demand) + correction_hat)`; it never sees the raw target, only how far a given cell and time bucket departs from its own history. The motivation is that `base_demand` already carries much of the predictable signal, so a model asked to reproduce it from scratch spends its capacity on something the pipeline has already computed, whereas the correction is the part that genuinely has to be learned. Since that correction has a far smaller scale and variance than `log1p(demand)`, hyperparameters tuned for one target do not transfer to the other, and the residual model is therefore tuned from scratch rather than inheriting the direct model's settings.

Both strategies are bounded by fitting cost. SVR is $O(n^2)$ to fit, which puts training splits of up to 6.2 million rows out of reach, so each level is fitted on a class-balanced sample: 4,000 rows per class for the kernel scan and 8,000 per class for the grid search, with selection scored on a 50,000-row validation subsample and only the level's winner re-scored on the full validation split. Balancing rather than uniform sampling is what makes such a small sample usable, since at `h3_7/hourly` roughly 94% of rows are zero and a uniform draw would be almost entirely empty

cells; where a level is not zero-inflated the zero class runs out and the sample tilts non-zero on its own. Validation and test are always evaluated at natural prevalence, so training and evaluation deliberately see different class mixes.

Tuning then proceeds in two stages per level and feature set. A kernel scan compares linear, polynomial and RBF at fixed $C=1$ and `gamma="scale"`, judged by validation R^2 in count space, and the winning kernel goes into a three-fold grid search over C in $\{1, 10, 50\}$ and `gamma` in $\{\text{scale}, 0.05, 0.2\}$, extended by `degree` in $\{2, 3\}$ where the polynomial kernel wins, scored by count-space MAE. Four feature sets compete at every level (a basic calendar, location and history block, then that block plus weather, plus POI, and plus both), with the winner taken on validation R^2 in count space. The direct notebook runs this through sklearn's `GridSearchCV`; the residual notebook reimplements the identical grid as a manual K-fold loop, because a standard scorer never sees a row's `base_demand` and so cannot turn a predicted correction back into a trip count. LIBSVM's iteration count is capped, as some C/gamma combinations otherwise run without converging.

4.2.2.1 Validation Results

Table 1: Direct SVR (3.2a) vs. residual-on-baseline SVR trained from scratch (this notebook), per level, on the validation split.

	spatial	temporal	feature_set	kernel	skill_MAE_direct	skill_MAE_residual	delta_
0	h3_7	hourly	basic+weather	linear	-0.1125	0.0026	0.1151
1	community	daily	basic+weather	linear	-0.0347	0.0534	0.0881
2	community	hourly	basic+poi+weather	rbf	-0.0844	-0.0098	0.0746
3	census	daily	basic+poi+weather	rbf	-0.0646	0.0029	0.0675
4	census	hourly	basic	linear	-0.3547	-0.3214	0.0333
5	h3_7	daily	basic+poi	linear	0.0019	0.0122	0.0103

The direct model does not beat the historical mean it was given as a feature. At five of six levels its tuned winner is worse than the baseline on validation MAE, from -3.5% at community/daily down to -35.5% at census/hourly, and the single level it wins, h3_7/daily, it wins by 0.2%, which is a tie in all but sign. Predicting the correction instead improves the skill score at all six levels, but the improvement is uneven and mostly modest: community/daily is the one clear win at +5.3% ($R^2 = 0.970$), h3_7/daily turns its tie into a slim +1.2%, h3_7/hourly (+0.3%) and census/daily (+0.3%) remain ties, and community/hourly stays just short of the baseline at -1.0%.

The exception is census/hourly, the sparsest grid at 96.5% zero rows, which loses heavily under both strategies (-35.5% direct, -32.1% residual). Its error profile is diagnostic: MAE skill is deeply negative while RMSE skill is roughly flat (+0.7% residual). A model trained on a balanced sample and scored at natural prevalence predicts a little too much demand across a very large mass of empty cells, which accumulates in MAE but barely registers in an RMSE dominated by the few large errors at busy cells.

Two patterns hold across the grid. Daily aggregation beats hourly at all three spatial resolutions under both strategies, consistent with hourly bins being sparser and noisier. And kernel selection splits cleanly between the strategies: every direct winner is RBF, while four of six residual winners are linear (both h3_7 levels, census/hourly and community/daily). This is a property of the transform rather than a preference for simplicity. A linear model in log space is unbounded, and `expm1` turns a moderately wrong log prediction into an enormous count,

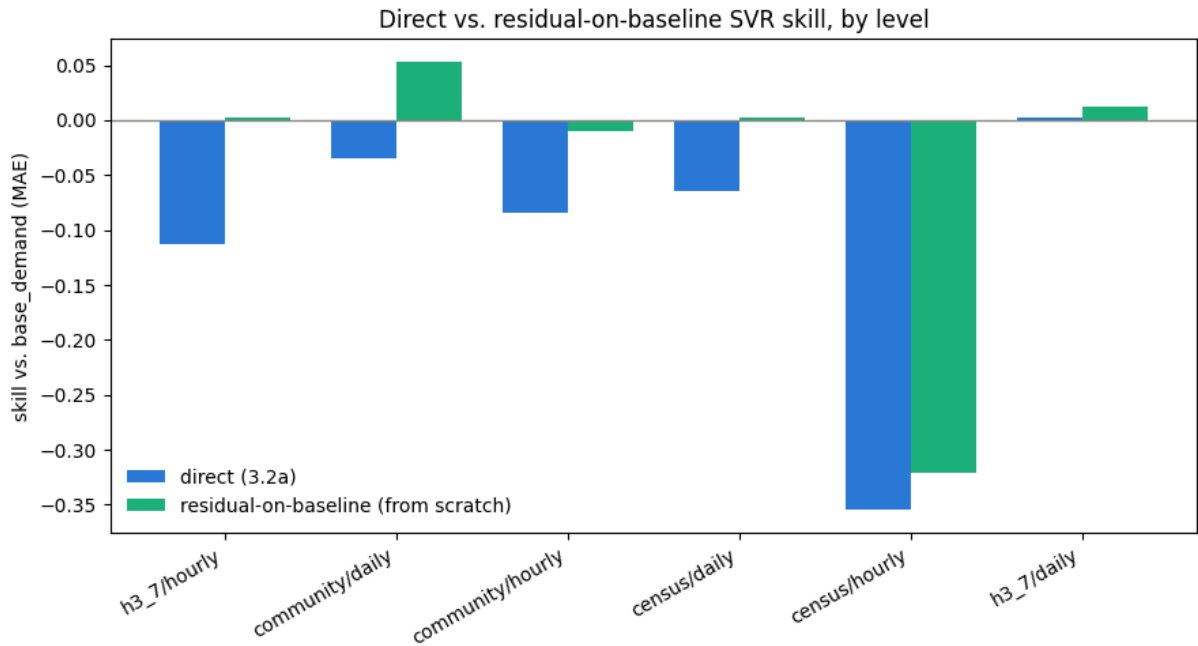


Figure 1: Direct (3.2a) vs. residual-on-baseline (this notebook) SVR skill, both independently tuned, per level.

so the direct target needs a bounded nonlinear fit; the residual’s small scale never pushes the reconstruction into that regime. All six residual winners also settled at $C=1$, against direct winners spread from $C=1$ to $C=50$. [GAP: $C=1$ is the lowest value in the search grid, so these winners sit on a grid edge. Either extend the grid downwards or confirm that validation performance is flat there before reporting the setting as converged.]

The test split is untouched at this stage. It is scored once, for the selected models, in the comparison against the neural network.

4.2.2.2 Shortfalls and Improvement Levers

The plainest shortfall is how little either strategy gains on the historical mean. That is less a tuning failure than a consequence of the framing: with autoregressive signal ruled out, cell identity plus calendar sits close to the ceiling of what is predictable here, and a historical mean over those same keys is already close to optimal. It also bounds what the obvious remedy would buy. Each level is fitted on at most 16,000 rows because SVR’s quadratic fitting cost makes anything larger impractical, but more rows of the same features are unlikely to clear a ceiling set by the features themselves. The lever worth testing is how the sample is drawn rather than how much of it is used, and natural-prevalence training is the untested alternative that two separate results point towards: the census/hourly collapse described above, and the zero-demand behaviour below.

That second result is a specific and fixable failure mode. On the held-out comparison the residual model is worse than the baseline precisely on cells whose actual demand turns out to be zero, where the neural network is better; the correction term does not shrink to zero where it should. Because zero rows are between 79% and 94% of all rows depending on the level, that one segment carries disproportionate weight in any aggregate MAE. Two remedies are worth testing: a floor or clip on the reconstructed correction, or a two-stage gate that decides zero against non-zero before regressing magnitude at all. [GAP: the direction of this effect has held

across every version of the comparison so far, but the segment magnitudes in 3.4 are pending regeneration, since the FNN predictions there are stale. Do not quote segment skill figures until 3.3 and 3.4 have been re-run.]

One design shortcut needs its own note, because it looks like a limitation and probably is not. Hyperparameters are searched only within the kernel that won the scan, rather than running a full grid inside every kernel. An independent full per-kernel search exists in `models_predictions/SVR/residual_kernelgrid/` and was previously read as confirming the shortcut. It can no longer be read that way. Those artifacts date from before the zero-mile-trip drop, and the giveaway is the baseline rather than the models: at `h3_7/daily` the full-grid run records a baseline validation MAE of 14.658 against 15.830 in the current run, and the historical mean is a property of the data alone, not of the search. The two runs are therefore scored on different datasets, and on the current artifacts only two of six levels even select the same kernel and feature set. [GAP: re-run the full per-kernel grid search on the post-drop data, or drop the claim that the two-stage shortcut is validated. As it stands the check is untested against the data actually reported here. Note also that the `level_comparison.csv` this comparison is usually cited from does not exist under `residual_kernelgrid/`; only the per-level bundles do.]

Three improvement levers remain that do not reintroduce autoregressive signal. Weather enters the model as a single citywide hourly series, so it varies across hours but never across cells within an hour, which leaves both weather-by-hour and weather-by-location interactions unexplored, and makes finer or spatially resolved weather data worth acquiring. POI enters as counts per cell, where a distance to the nearest POI of each category would carry more information about why a cell generates trips. The permutation and drop-block analysis supports that ordering: after `base_demand`, the daily levels depend most on the POI block (+5.5% MAE at `h3_7/daily`, +9.4% at `census/daily`, +13.2% at `community/daily` when it is removed), while the hourly levels depend on history instead (+148%, +32% and +22% respectively). [GAP: those figures are the direct SVR’s permutation run in 3.2a; 3.2b’s residual-specific ablation was not regenerated this run and is currently unverified against the post-drop data.]

4.2.3 Deep Learning Model

4.2.4 Model Comparison

The SVM and the neural network are compared on the **validation** split at `h3_7/daily`; the held-out test set is scored once, at the end, for the model selected on validation. Below: predicted versus actual demand, and the spatial distribution of the prediction error across Chicago (mean absolute error, mean relative error, and the coefficient of determination per hexagon).

4.3 Smart Charging (Reinforcement Learning)

We frame the taxi driver’s home charging as a Markov Decision Process and solve it with Q-learning. The agent sets the charging power in 15-minute steps between the 2 p.m. arrival and 4 p.m. departure, trading off recharging cost against the risk of not meeting the (stochastic) energy demand for the working day.

4.3.1 Problem Formulation (MDP)

4.3.2 Environment (Discrete-Event Simulation)

4.3.3 Q-Learning Agent

4.3.4 Training

4.3.5 Learned Policy

The heatmap below shows the learned charging policy: for each battery level and timestep, the charging action the agent has learned to take.

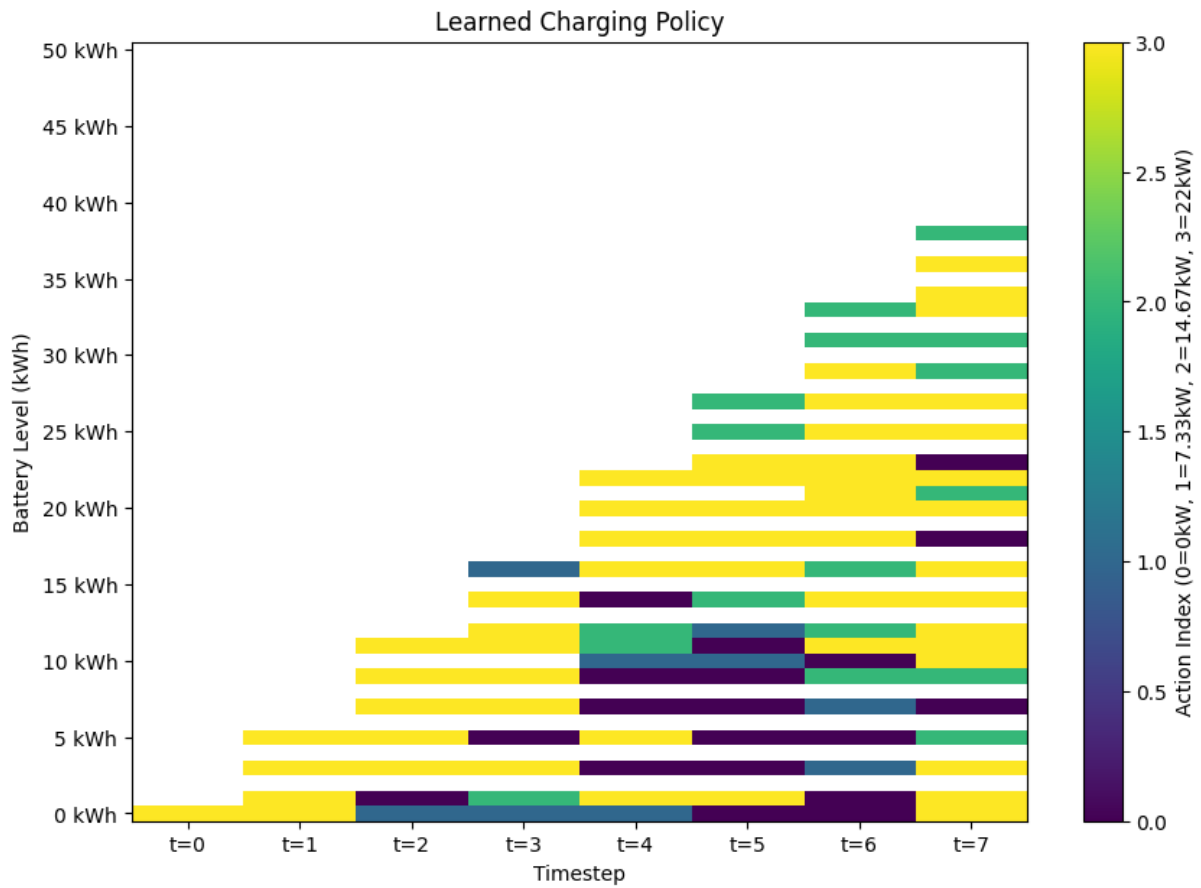


Figure 2: Learned Charging Policy

4.3.6 Evaluation

5 Discussion and Outlook

5.1 Summary of Findings

5.2 Implications for the Client

5.3 Limitations

5.4 Future Work

References